

Chapter 10: De Novo Protein & Antibody Design

Chapter 9 asked whether a model can predict the 3D structure a given sequence folds into. This chapter asks the inverse pair of questions: given a desired 3D *shape* — a backbone that would bind a chosen target, or simply hold together as a stable fold — what *sequence* should produce it, and can a backbone with that shape be generated in the first place, rather than found by luck in the Protein Data Bank? Splitting “de novo design” into exactly these two sub-problems — backbone generation (Section 10.1) and sequence design conditioned on a fixed backbone (Section 10.2) — is itself the field’s central architectural decision, and this chapter’s hands-on project (Section 10.4) works entirely on the second half of that split, for reasons its own feasibility investigation makes explicit before any code is run.

10.1 Backbone Generation

RFdiffusion (Watson et al., 2023) reframes backbone generation as a denoising diffusion process (the same generative principle Chapter 7 §7.4 introduced for 3D *molecular* structures) applied instead to protein *backbones*: a sequence of rigid-body frames, one per residue, of exactly the kind Chapter 9 §9.1 described AlphaFold2’s structure module producing as its *output*. RFdiffusion runs that relationship in reverse. Training corrupts real backbones by progressively randomizing each residue’s frame — rotation and translation both — toward an uninformative distribution over $SE(3)$ (the group of rigid 3D motions), and a network built on RoseTTAFold’s own architecture (Chapter 9 §9.2) learns to reverse that corruption one small step at a time. Sampling then starts from pure noise — frames scattered with random orientation and position — and iteratively denoises them into a physically plausible backbone, either completely unconditionally (a novel fold with no target shape specified at all) or *conditioned* on a fixed motif: a target protein’s binding surface, held fixed throughout sampling, around which the rest of the backbone is generated to make contact — exactly the “binder design” mode of most direct relevance to this chapter’s own hands-on target-binding peptide problem, and to Chapter 11’s docking-based validation of designed binders. Watson et al. (2023) validate designs from every one of these modes experimentally (not just computationally): x-ray crystallography and cryo-EM structures of expressed, purified designed proteins matching the intended generated backbones.

A feasibility investigation, before any code was written. Before committing to RFdiffusion for this chapter’s hands-on project, its real installation requirements were checked directly against the current RosettaCommons/RFdiffusion GitHub repository rather than assumed. Three things ruled it out for this book’s runtime target (a free-tier Google Colab CPU/GPU instance, or the CPU-only ~16 GB RAM development environment this book’s code is authored and tested in): (1) it depends on NVIDIA’s SE(3)-Transformer implementation, installed from source via a conda environment pinned to CUDA 11.1-era package versions that need per-user customization to run on newer GPUs, not a pip-installable package; (2) its own README states setup “should take less than 30 minutes on a standard desktop computer” assuming that CUDA/conda stack already resolves cleanly, which is a real, nontrivial dependency-resolution risk on a from-scratch install, let alone a CPU-only one; and (3) inference requires downloading multiple real model checkpoints (Base_ckpt.pt, Complex_base_ckpt.pt, Complex_Fold_base_ckpt.pt, and others) from files.ipd.uw.edu, collectively several gigabytes, for a diffusion sampling process whose

per-step compute cost — dozens of forward passes through a RoseTTAFold-scale network per generated backbone — assumes GPU acceleration throughout; a CPU fallback exists in principle but was not something this book’s authoring environment could validate in reasonable time. This is the same category of constraint Chapter 9 §9.4 hit with the full local `esmfold_v1` checkpoint, but here there is no equivalent lightweight public inference API standing in for it the way the ESM Metagenomic Atlas API did for ESMFold — RFdiffusion has no comparably simple hosted endpoint, only a Colab notebook (itself requiring a GPU runtime) and a Docker image (requiring the same underlying CUDA stack). Section 10.1 therefore covers RFdiffusion as theory only; Section 10.4’s hands-on project starts from real, already-solved backbones (Chapter 9’s bundled 1UBQ, and this chapter’s own 1YCR) rather than generating new ones, and says so explicitly rather than silently substituting one for the other.

10.2 Sequence Design (Inverse Folding)

Whether a backbone comes from RFdiffusion, from a real crystal structure, or from any other source, it is not yet a protein: an amino acid sequence still has to be chosen that will actually fold into that exact shape. This is **inverse folding** — literally the inverse of Chapter 9’s own subject, predicting sequence from structure rather than structure from sequence — and **ProteinMPNN** (Dauparas et al., 2022) is this chapter’s hands-on model for it.

Given a fixed backbone ($C\alpha$, N, C, and O coordinates for every residue, no side chains), ProteinMPNN represents it as a graph — each residue a node, edges connecting each residue to its k nearest spatial neighbors ($k=48$ in the checkpoint this chapter uses) — and processes it with a message-passing encoder structurally similar in spirit to Chapter 6 §6.3’s MPNN, updating each node’s representation from its geometric neighborhood alone (distances, orientations — never a specific amino acid identity, since none exists yet). A decoder then generates the sequence autoregressively, one residue at a time, each step conditioned on the encoder’s structural embeddings and every previously decoded residue,

$$p_{\theta}(s_1, \dots, s_L \mid \text{backbone}) = \prod_{i=1}^L p_{\theta}(s_i \mid s_{<i}, \text{backbone})$$

— the same autoregressive factorization Chapter 7 §7.3 used for *generating* molecules from nothing, here instead generating a sequence *conditioned on* a fixed 3D structure it must actually fit. Critically, the decoding order is not fixed left-to-right: at both training and inference time it is randomly permuted per sample, so no position is structurally privileged as “first” or “last,” a design choice Dauparas et al. (2022) report improves robustness precisely because a real backbone imposes no left-to-right ordering on which residues constrain which others. Multiple chains can be decoded jointly with some marked fixed (their real sequence supplied and never resampled) and others marked for design — the officially supported mechanism this chapter’s own binder-only redesign experiment uses directly, letting a target protein’s real sequence stay fixed while only a binding partner’s sequence is redesigned for a shared, fixed backbone.

Dauparas et al. (2022) report ProteinMPNN reaches 52.4% **native sequence recovery** averaged across their benchmark set of real protein backbones — i.e., on average, just over half of the amino acids ProteinMPNN independently chooses for a real backbone match the real, naturally evolved

sequence that backbone actually has — compared with 32.9% for Rosetta, the physics-based design method it was benchmarked against. Section 10.4 measures this same statistic directly, on one real backbone, rather than only citing the published aggregate.

10.3 Antibody & Peptide Engineering

Sections 10.1-10.2’s general backbone-generation-then-sequence-design pipeline extends directly to two design problems of particular pharmaceutical relevance: designing short peptides that bind a chosen target with high affinity, and designing the complementarity-determining regions (CDRs) of antibodies — the hypervariable loops that make essentially all of an antibody’s contact with its antigen — rather than an entire fold from scratch.

High-affinity peptide binders. Vázquez Torres et al. (2024) apply RFdiffusion (with a helix-scaffolding variant tuned specifically for short, often partially disordered target peptides) plus ProteinMPNN sequence design plus AlphaFold2-based filtering (Chapter 9 §9.1) to design novel binders against several bioactive helical peptide targets, validating designs experimentally with binding affinities in the nanomolar range for some targets — a direct, real precedent for Section 10.4’s own much smaller-scale target-binding peptide exercise, and a concrete illustration of the three-stage pipeline (generate, design, filter) this book’s chapters 9-11 build toward in combination.

Antibody CDR design. Antibodies pose a structurally distinct version of the same problem: the CDR loops occupy a small, well-defined region of an otherwise conserved immunoglobulin scaffold, so design methods built for general backbone generation need adaptation to that specific geometry. Cutting et al. (2025) train a dedicated $SE(3)$ diffusion model directly on antibody variable-domain structures — the same rigid-frame diffusion principle as RFdiffusion (Section 10.1), specialized to the CDR-loop generation problem rather than applied to general backbones — reporting designed loops with realistic length and conformational diversity relative to real antibody repertoires. Neither this specialized antibody model nor RFdiffusion’s own general binder-design mode is run in this chapter’s hands-on project, for the same feasibility reasons detailed in Section 10.1; both are covered here as the theoretical bridge from single-peptide binder design (Section 10.4’s own scope) to the antibody-specific design problem, a natural next extension this chapter does not itself attempt.

10.4 Hands-on Project: ProteinMPNN Sequence Design, Validated Against Real Ground Truth

The project code lives in this chapter’s folder (ch10_protein_design/). Given Section 10.1’s feasibility finding, this project starts from **real, already-solved backbones** rather than RFdiffusion-generated ones, and puts ProteinMPNN’s design output through two different kinds of real validation.

Real data: two real backbones

PDB 1UBQ (Vijay-Kumar et al., 1987): human ubiquitin, 76 residues, 1.8-Å resolution — the same real crystal structure Chapter 9 validated ESMFold’s confidence against, reused here for direct

continuity (data/1UBQ.pdb, refetched fresh into this chapter’s own folder per this book’s per-chapter self-containment convention).

PDB 1YCR (Kussie et al., 1996): the MDM2 oncoprotein (chain A, 85 resolved residues) bound to the p53 transactivation-domain peptide (chain B, 13 resolved residues, real sequence ETFSDLWKLLPEN, PDB residue numbers 17-29 — two further N-terminal residues, 15-16, are present in the construct but unresolved in the electron density and excluded here along with everything else the crystal structure does not actually determine). This is a real, extensively characterized drug-discovery-relevant interface: MDM2 negatively regulates the p53 tumor suppressor by binding this exact peptide region, and the small-molecule “Nutlin” class of MDM2 inhibitors was designed specifically to occupy this same binding cleft — this chapter’s peptide-redesign exercise targets the same real interaction from the opposite side, designing new peptides for the site rather than small molecules to block it.

ProteinMPNN wraps the real official model

protein_design.py vendors the real, official ProteinMPNN source (third_party/proteinmpnn/protein_mpn_utils.py, MIT-licensed, unmodified — see third_party/proteinmpnn/NOTICE.md) and the real pretrained v_48_020 checkpoint (~6.7 MB; 48 nearest-neighbor edges, 0.20 Å training noise — the model’s own default). No weights are retrained or modified.

```
def design_sequences(pdb_path, model, designed_chains=None, fixed_chains=None,
                    temperature=0.1, num_samples=1, seed=0):
    pdb_dict_list = parse_PDB(str(pdb_path), ca_only=False)
    name = pdb_dict_list[0]["name"]
    chain_id_dict = None
    if designed_chains is not None:
        chain_id_dict = {name: (designed_chains, fixed_chains or [])}
    ...
    X, S, mask, ..., chain_M, chain_M_pos, ... = tied_featurize(batch, device,
chain_id_dict)
    ...
    sample_dict = model.sample(X, randn, S, chain_M, ...,
temperature=temperature,
                             chain_M_pos=chain_M_pos, ...)
```

A real bug surfaced immediately when this function was first tested on the two-chain 1YCR structure: parse_PDB keys its internal structure dictionary by the *parsed* structure name (the PDB file’s own name field), not by the caller-supplied file path — passing a hand-built chain_id_dict keyed by a guessed name (“1YCR”, or worse, the full input file path) raises a KeyError the first time chain-restricted design is attempted. Fixed by building chain_id_dict internally, from the real name parse_PDB itself returns, rather than assumed by the caller — the version shown above, and the version this chapter’s real results were computed with.

Experiment 1: whole-chain redesign of ubiquitin

Redesigning all 76 positions of the real 1UBQ backbone at three sampling temperatures (5 independent samples each, following the same real-parameter-sweep pattern Chapters 8-9 use), native sequence recovery against the real wild-type ubiquitin sequence came out as follows:

Temperature	Mean recovery	Std (n=5)
0.1	56.3%	1.0%
0.2	56.8%	1.3%
0.3	54.7%	3.2%

All three land close to — and here, on this one real backbone, slightly above — Dauparas et al.’s (2022) published 52.4% cross-benchmark average (Section 10.2), with variance visibly increasing at the highest sampling temperature, exactly as expected: higher temperature samples more diversely away from the model’s single most-likely sequence per position, so agreement with the one real wild-type sequence should both drop somewhat and vary more from sample to sample, which is precisely what happened.

Structural self-consistency, attempted via ESMFold. A design being close to the real wild-type sequence is one kind of validation; a second, independent kind (standard practice in the actual design literature, not unique to this chapter) is to fold the *redesigned* sequence with a structure predictor unrelated to ProteinMPNN and check whether it lands back near the same backbone the design was conditioned on. As a positive control, first folding the real native ubiquitin sequence via the live ESM Metagenomic Atlas API (Chapter 9’s method, reused verbatim) succeeded in the same few seconds as Chapter 9’s own run, landing at the identical **0.827 Å C α RMSD** against the real 1UBQ structure (deterministic inference, same sequence, same model — an exact match is the expected, correct outcome, not a coincidence). Folding the *redesigned* sequence — a real, independently generated 76-residue sequence with roughly 55% identity to the native one, not random noise — was then attempted the same way, and the API returned an HTTP 504 Gateway Timeout, reproducibly, across every redesigned sequence tested (four independent samples, spanning all three sampling temperatures above). This is reported as a real finding, not omitted: it is not a general property of 76-residue sequences (the real native sequence at the same length folds in seconds), and Chapter 9 already established, separately, that this same API times out on sequences that are genuinely disordered. A designed sequence being structurally plausible by construction — locally consistent with the backbone ProteinMPNN conditioned on at each position — does not guarantee it sits confidently within ESMFold’s own training distribution of natural, evolutionarily selected sequences; a sequence unlike anything the folding model has seen may require more internal refinement to reach a confident answer than this particular free, rate-limited inference endpoint’s fixed timeout allows, a distinct real failure mode from “folds quickly to something wrong.” This is the honest outcome of a real, repeated attempt, not a claim that ProteinMPNN’s design failed — Experiment 1’s actual design-quality evidence is the real sequence-recovery numbers above, which do not depend on this API succeeding.

Experiment 2: real target-binding peptide redesign

Redesigning only chain B of the real 1YCR complex (the 13-residue p53 peptide), with chain A (MDM2, 85 residues) held fixed as real structural context — ProteinMPNN’s officially supported multi-chain “which chains to design” mechanism, this chapter’s real substitute for an RFdiffusion-generated binder backbone — produced, across five independent samples at temperature 0.2:

Designed sequence	Overall recovery	Hot-spot recovery (F19/W23/L26)	Non-hot-spot recovery
ETFEEIWAKLPQS	46.2%	100%	30.0%
ETFEELWSQLPQS	53.8%	100%	40.0%
ETFEELWSKLPQS	53.8%	100%	40.0%
ETFEELWSKLPQS	53.8%	100%	40.0%
MTFEELWALLPQS	53.8%	100%	40.0%

(Native peptide: ETFSDLWKLLPEN.)

This is the chapter’s central real finding, and it is worth reading precisely. Overall sequence identity to the native peptide is modest (46-54%) — ProteinMPNN is not simply reproducing p53’s real sequence. But at exactly three positions — the real, literature-verified Phe19/Trp23/Leu26 hydrophobic triad Kussie et al. (1996) report inserting deep into MDM2’s binding cleft (PDB residue numbers verified directly against the bundled 1YCR.pdb, not assumed from the paper text) — every one of five independent designs recovers the exact native residue, a **100% hot-spot recovery rate** against a **38.0%** average recovery rate at every other position. This is not coincidental: those three side chains are the ones making buried, shape-complementary contacts against MDM2’s surface, so the fixed backbone geometry structurally constrains what can occupy those three positions far more tightly than it constrains the peptide’s solvent-exposed face — precisely the kind of structure-function coupling a purely backbone-conditioned model has no access to except through the geometry itself, since ProteinMPNN is never told which positions matter functionally. Recovering a real, independently published pharmacological hot-spot from geometry alone is a substantially stronger validation of design quality than bulk sequence identity would be on its own.

Why the peptide’s ESMFold check reports a timeout. Folding the isolated 13-residue native peptide alone (no MDM2 present) via the live API was attempted the same way as Experiment 1, and — like every redesigned variant also tested — it reproducibly returned an HTTP 504 Gateway Timeout, confirmed twice for the native sequence specifically. This was checked against a real, independent positive control at comparable length: a 20-residue sequence of Trp-cage (a well-characterized, independently stable miniprotein) folds via the same API in a few seconds, ruling out sequence length alone as the cause. The most likely real explanation, consistent with Chapter 9’s own finding about the free-standing p53 transactivation domain: this peptide is disordered in isolation and only adopts the ordered α -helical conformation the crystal structure captures upon binding MDM2 (Kussie et al., 1996 describe this bound conformation explicitly) — the same coupled folding-and-binding phenomenon Chapter 9 discussed for the adjacent p53 transactivation-domain region, now encountered again from the design side. A structure predictor asked to fold a sequence with no single stable free conformation may need more iterative refinement to converge than this endpoint’s timeout allows; in that light, the timeout is itself indirect, real evidence for exactly the biological behavior the crystal structure was solved to capture, not merely an inconvenience to route around.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `numpy>=1.24`, `scipy>=1.10`, `biopython>=1.81`, `requests>=2.28` in `requirements.txt`, validated against `torch 2.13.0`, `numpy 2.5.2`, `scipy 1.18.0`, and `biopython 1.88` on Python 3.12). The real ProteinMPNN model code and checkpoint are bundled directly (`third_party/proteinmpnn/`, `proteinmpnn_weights/v_48_020.pt`), so `python protein_design.py --skip-esmfold` reproduces both design experiments fully offline and deterministically (verified directly: identical seeds reproduce byte-identical designed sequences). The 16-test suite in `tests/test_protein_design.py` runs the real model directly against the real bundled 1UBQ.pdb/1YCR.pdb structures — ProteinMPNN’s checkpoint is small and fast enough on CPU (~1 second per design call) that, unlike Chapters 8-9’s much larger protein language models, no synthetic/tiny substitute model is needed for testing; only the ESMFold network call is mocked, except one test that calls the live API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 16 results.

Limitations and what comes next

This project validates ProteinMPNN’s design quality on two real backbones with two different, targeted kinds of evidence (native recovery against real ground truth; real, literature-verified hot-spot recovery on a real drug-relevant interface) rather than a broad benchmark across many folds, which is Dauparas et al.’s (2022) own published scope, not repeated here. It does not generate novel backbone geometry at all — Section 10.1’s feasibility finding rules that out for this environment — so “de novo” here means a real sequence newly designed for a real, existing backbone, not a wholly novel backbone. ESMFold structural validation succeeded for natural sequences and did not complete for either designed or naturally disordered ones, a real, reported limitation of this particular free API rather than of ProteinMPNN’s designs themselves. Chapter 11 picks up the natural next question this chapter’s own peptide-redesign result raises directly: now that a real binder sequence has been designed against a real target interface, does it actually dock into that same site the way physics-based and deep-learning docking methods predict?

A note on Google Colab

`torch`, `numpy`, `scipy`, and `requests` are preinstalled on Colab’s default runtime; only `biopython` needs `!pip install biopython`. No GPU is required for ProteinMPNN inference at this scale — a full sampling run over either backbone in this chapter completes in well under a minute on a free CPU-only Colab instance.

References

- Watson, J. L., Juergens, D., Bennett, N. R., Trippe, B. L., Yim, J., Eisenach, H. E., Ahern, W., Borst, A. J., Ragotte, R. J., Milles, L. F., Wicky, B. I. M., Hanikel, N., Pellock, S. J., Courbet, A., Sheffler, W., Wang, J., Venkatesh, P., Sappington, I., ... Baker, D. (2023). De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976), 1089-1100. <https://doi.org/10.1038/s41586-023-06415-8>
- Dauparas, J., Anishchenko, I., Bennett, N., Bai, H., Ragotte, R. J., Milles, L. F., Wicky, B. I. M., Courbet, A., de Haas, R. J., Bethel, N., Leung, P. J. Y., Huddy, T. F., Pellock, S., Tischer, D., Chan, F., Koepnick, B., Nguyen, H., Kang, A., ... Baker, D. (2022). Robust deep learning-based protein

sequence design using ProteinMPNN. *Science*, 378(6615), 49-56. <https://doi.org/10.1126/science.add2187>

- Vázquez Torres, S., Leung, P. J. Y., Venkatesh, P., Lutz, I. D., Hink, F., Huynh, H.-H., Becker, J., Yeh, A. H.-W., Juergens, D., Bennett, N. R., Hoofnagle, A. N., Huang, E., MacCoss, M. J., Expósito, M., Lee, G. R., Bera, A. K., ... Baker, D. (2024). De novo design of high-affinity binders of bioactive helical peptides. *Nature*, 626(7998), 435-442. <https://doi.org/10.1038/s41586-023-06953-1>
- Cutting, D., Dreyer, F. A., Errington, D., Schneider, C., & Deane, C. M. (2025). De Novo Antibody Design with SE(3) Diffusion. *Journal of Computational Biology*, 32(4), 351-361. <https://doi.org/10.1089/cmb.2024.0768>
- Kussie, P. H., Gorina, S., Marechal, V., Elenbaas, B., Moreau, J., Levine, A. J., & Pavletich, N. P. (1996). Structure of the MDM2 Oncoprotein Bound to the p53 Tumor Suppressor Transactivation Domain. *Science*, 274(5289), 948-953. <https://doi.org/10.1126/science.274.5289.948>
- Vijay-Kumar, S., Bugg, C. E., & Cook, W. J. (1987). Structure of ubiquitin refined at 1.8 Å resolution. *Journal of Molecular Biology*, 194(3), 531-544. [https://doi.org/10.1016/0022-2836\(87\)90679-6](https://doi.org/10.1016/0022-2836(87)90679-6)

See Chapter 9's references for Jumper et al. (2021, AlphaFold2's structure-module frames, the rigid-body representation RFDiffusion's own diffusion process operates over) and Lin et al. (2023, ESMFold), reused directly in Section 10.4's validation. RDKit, DeepChem, XGBoost, and PyTorch Geometric, used throughout Chapters 1-7, play no direct role in this chapter. `third_party/proteinmpnn/protein_mpnn_utils.py` is the real, official ProteinMPNN source code, vendored verbatim under its MIT license (`third_party/proteinmpnn/LICENSE`) — see `third_party/proteinmpnn/NOTICE.md` for full attribution.

All recovery rates, RMSD values, and hot-spot statistics cited in Section 10.4 were computed directly by running `protein_design.py` against the real bundled PDB structures and the real ProteinMPNN checkpoint on 2026-08-20, not taken from a secondary source — see `results/protein_design_results.json` to reproduce.